

データベース仕様書

サービス名: 決済メール解析による自動家計簿サービス 言語 / 想定 DB: Go / PostgreSQL (GORM)

ドラフト版。メンタリングでの相談を前提とする。

1. データベース概要

決済通知メールから自動抽出した支出データを管理するデータベース。

生のメール (`email_messages`) → メールから抽出した決済イベント (`payment_events`) → 名寄せ後の実取引 (`transactions`) という **3層構造**を持つ。生データを保持することで、抽出・名寄せロジックを変更した際に全件を再計算できる。

あわせて、店舗マスタと表記ゆれの吸収 (`merchants / merchant_aliases`)、カテゴリの自動分類 (`categories / category_rules`)、通知 (`notification_settings / notifications`)、サブスク・予算・月次サマリを扱う。

共通方針

方針	内容
マルチテナント	全業務テーブルに <code>user_id</code> を持ち、ユーザ間でデータを分離する
金額	<code>bigint</code> の最小単位 (円は 1 = 1円)。浮動小数点は使用しない
通貨	<code>currency</code> (ISO 4217) + 円換算額 <code>amount_jpy_minor</code> を併記
日時	<code>timestampz</code> で保持。集計は JST 基準
主キー	<code>bigint</code> の自動採番
共通列	原則すべてのテーブルに <code>created_at / updated_at</code>
削除	原則は物理削除。例外は 2 つで、 <code>transactions</code> は <code>deleted_at</code> による論理削除 (3.7)、メール連携の解除は <code>mail_accounts.status = 'disabled'</code> (3.2)。それぞれ理由を各節に記載する

2. テーブル一覧

コア (Step3 で実装)

No.	テーブル名	概要	対応工程
1	<code>users</code>	アプリ利用者	P0
2	<code>mail_accounts</code>	連携したメールアカウントと認証情報	P0

No.	テーブル名	概要	対応工程
3	sync_jobs	メール取得ジョブの実行履歴	P1
4	email_messages	取得したメールの生データ	P1 / P2
5	parser_templates	送信元・件名の判定条件と抽出ルール	P2 / P3
6	payment_events	メールから抽出した決済イベント（名寄せ前）	P3 / P4 / P4.5
7	transactions	名寄せ後の実取引（1回の支出 = 1行）	P5 / P6
8	merchants	正規化済み店舗マスタ（支店単位・位置情報付き）	P4 / P4.6
9	merchant_aliases	表記ゆれ → merchants のマッピング	P4
10	payment_methods	決済手段マスタ	P4
11	categories	カテゴリマスタ	P6
12	category_rules	カテゴリ自動分類ルール	P6
13	notification_settings	通知設定（チャンネル・閾値・静音時間帯）	P4.5 / P8
14	notifications	通知の送信履歴	P4.5 / P8

拡張（余力があれば）

No.	テーブル名	概要	対応工程
15	subscriptions	検出したサブスク	P7
16	budgets	予算設定	P8
17	monthly_summaries	月次サマリの生成・配信記録	P8

3. 各テーブルの詳細

3.1 users

アプリの利用者。認証は Google OAuth に委譲するため、パスワードは保持しない。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ユーザ ID
google_sub	varchar(255)	○	UNIQUE	Google アカウントの一意識別子（sub クレーム）
email	varchar(255)	○	UNIQUE	メールアドレス
display_name	varchar(100)			表示名

カラム名	型	必須	制約	説明
timezone	varchar(50)	○	default Asia/Tokyo	集計の基準タイムゾーン
created_at	timestampz	○		作成日時
updated_at	timestampz	○		更新日時

インデックス

追加のインデックスは定義しない。ログイン時の本人特定に使う `google_sub` の検索は、`UNIQUE` 制約に対して PostgreSQL が自動生成する一意インデックスで賄える。

3.2 mail_accounts

連携したメールアカウント。取込方式を `provider` で切り替えられるようにし、Gmail API / GAS / IMAP のいずれを選んでも同じテーブルで扱えるようにする。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
email_address	varchar(255)	○		連携したメールアドレス
provider	varchar(20)	○		取込方式 (<code>gmail_api</code> / <code>gas</code> / <code>imap</code>)
credential_encrypted	text	○		暗号化した認証情報（リフレッシュトークン等）
credential_expires_at	timestampz			認証情報の有効期限
sync_cursor	varchar(255)			前回同期位置（Gmail の historyId 等）
backfilled_until	timestampz			バックフィル完了済みの起点
last_synced_at	timestampz			最終同期日時。同期が成功したら 0 件でも更新する（死活監視に使うため。3.3 参照）
status	varchar(20)	○	default active	<code>active</code> / <code>reauth_required</code> / <code>disabled</code> （連携解除はこの値で表現し、行は削除しない。下記の削除方針を参照）
created_at	timestampz	○		作成日時
updated_at	timestampz	○		更新日時

制約

- `UNIQUE(user_id, email_address)`: 同じアドレスの二重連携を防ぐ

インデックス

インデックス名	対象カラム	用途
idx_mail_accounts_status	status	バッチが同期対象アカウントを抽出

ユーザの連携一覧取得 (`WHERE user_id = ?`) は、`UNIQUE(user_id, email_address)` の自動生成インデックスの左端プレフィックスで賄えるため、専用インデックスは定義しない。

削除方針

「連携の解除」と「アカウントの退会」は**別の操作**として扱う。

操作	実行内容	影響
連携の解除	<code>status = 'disabled'</code> に更新し、 <code>credential_encrypted</code> を空にする	同期は停止するが、 <code>email_messages</code> 以降のデータはすべて残る
アカウントの退会	<code>DELETE FROM users WHERE id = ?</code>	FK の CASCADE により、そのユーザの全データが連鎖削除される

理由: `email_messages.mail_account_id` は CASCADE のため、連携解除で `mail_accounts` の行を物理削除するとメールの生データまで消え、「全件再計算の起点を保持する」という 3.4 の方針が成立しなくなる。ユーザが連携解除に期待するのは「これ以上メールを読まないでほしい」であって、「これまでの家計簿を消してほしい」ではないため、行は残す設計とした。

一方で「もう読まないでほしい」という意図は満たす必要があるため、**解除時に `credential_encrypted` を空にし、リフレッシュトークンを保持しない。**

3.3 sync_jobs

メール取得ジョブの実行履歴。可観測性（無音の故障を検知する）のために必須。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
mail_account_id	bigint	○	FK → mail_accounts.id (CASCADE)	対象アカウント
trigger	varchar(20)	○		<code>scheduled</code> / <code>manual</code> / <code>backfill</code> / <code>push</code> (Push 通知に移行した場合に使用)
started_at	timestampz	○		開始日時
finished_at	timestampz			終了日時

カラム名	型	必須	制約	説明
status	varchar(20)	○		running / success / partial / failed
fetches_count	integer	○	default 0	取得したメール数
parsed_count	integer	○	default 0	抽出に成功した数
failed_count	integer	○	default 0	抽出に失敗した数
cursor_before	varchar(255)			実行前のカーソル
cursor_after	varchar(255)			実行後のカーソル
error_message	text			エラー内容
created_at	timestampz	○		作成日時

インデックス

インデックス名	対象カラム	用途
idx_sync_jobs_account_started	(mail_account_id, started_at DESC)	同期履歴画面の表示

運用方針

#	方針	内容
1	記録 粒度	1分間隔で実行するため、そのまま記録すると1日1440行になる。大半は「新着なし」で記録する価値がないため、 fetches_count = 0 かつ status = success の実行は sync_jobs に記録しない
2	死活 監視	実行が成功したら 0 件でも必ず mail_accounts.last_synced_at を更新する（下記）
3	保持 期間	90日を超えた行は定期削除する。ただし status IN ('failed', 'partial') の行は障害分析のため保持する

方針2の理由: 方針1だけだと、「新着がなかった」と「バッチ自体が停止していた」がどちらも「**sync_jobs**に行かない」状態になり区別できない。**sync_jobs**は無音の故障を検知するためのテーブルなので、これでは本来の目的を果たせない。

成功時に**last_synced_at**を更新しておけば、行を増やさずUPDATE1回で最終成功時刻が保てる。死活監視は次のクエリで行い、該当行があればバッチ停止と判断する。

```
SELECT id, email_address, last_synced_at
FROM mail_accounts
WHERE status = 'active'
      AND last_synced_at < now() - interval '5 minutes';
```

設計判断: メール取得をポーリングにした理由

Gmail には Push 通知 (`users.watch` + Cloud Pub/Sub) があり、遅延 1〜2 秒・空振りゼロで、本来はこちらが適切である。それでも初期実装をポーリングにしたのは次の理由による。

観点	判断
クォータ	差分取得は受信箱の全走査ではなく <code>users.history.list</code> で行うため、1 日 1440 回でも消費は数千クォータ単位にとどまり、プロジェクト日次上限に対して無視できる
遅延	最悪 60 秒。カード会社が速報メールを送信するまでに数十秒〜数分かかるため、経路全体で見れば差は小さい
追加の運用コスト	Push には Pub/Sub の構築、公開 HTTPS エンドポイント、 7 日で失効する <code>watch</code> の更新バッチ が必要になる。本課題の主題（決済メールの解析）と関係のない運用コストが増える
信頼性	Pub/Sub は at-least-once のため、Push に移行しても取りこぼし対策の低頻度ポーリングは併用することになる

Push へ移行してもスキーマ変更はほぼ不要である。 Push が通知するのも `historyId` であり、`mail_accounts.sync_cursor` の意味は変わらない。「取得のトリガ」と「取得後の処理」を分離しているため、`sync_jobs.trigger` に `push` を追加するだけで両方式を同一テーブルで扱える。

移行の判断基準: 即時通知の遅延がユーザ体験上の課題になった時点。

3.4 email_messages

取得したメールの生データ。**通常の運用では削除しない。** 後段（抽出・正規化・名寄せ）のロジックを変更した際に全件を再計算する起点となる。メール連携を解除しても削除されない（3.2 の削除方針を参照）。

なお `transactions` はこのテーブルを参照せず自前のフィールドを持つため、プライバシー上の要請でこのテーブルを削除しても**家計簿データそのものは失われない**（失われるのは再計算とトレーサビリティのみ）。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
mail_account_id	bigint	○	FK → mail_accounts.id (CASCADE)	取得元アカウント
provider_message_id	varchar(255)	○		Gmail の message ID
thread_id	varchar(255)			スレッド ID
from_address	varchar(255)	○		送信元アドレス
from_name	varchar(255)			送信者名

カラム名	型	必須	制約	説明
subject	text			件名
received_at	timestampz	○		受信日時
body_text	text			本文（プレーンテキスト）
body_html	text			本文（HTML）
classification	varchar(20)	○	default unknown	payment / not_payment / unknown
classified_at	timestampz			判定日時
matched_template_id	bigint		FK → parser_templates.id (SET NULL)	マッチしたテンプレート
parse_status	varchar(20)	○	default pending	pending / success / failed / skipped
parse_error	text			抽出失敗の内容
created_at	timestampz	○		作成日時
updated_at	timestampz	○		更新日時

制約

- **UNIQUE(mail_account_id, provider_message_id)** ← 取込の冪等性を担保する中心的な制約。同じメールを再取得しても重複行が増えない。

インデックス

インデックス名	対象カラム	用途
idx_email_messages_user_received	(user_id, received_at DESC)	受信一覧の表示
idx_email_messages_queue	(user_id, classification, parse_status)	未判定・未処理メールの抽出（P2/P3 のキュー）
idx_email_messages_from	from_address	送信元ごとの受信件数集計（途絶検知）

3.5 parser_templates

送信元・件名から「決済メールかどうか」を判定し、本文から各項目を抽出するためのルール。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint		FK → users.id (CASCADE)	NULL の場合は全ユーザ 共通テンプレート
name	varchar(100)	○		テンプレート名（例: 楽天 カード 利用速報）
from_pattern	varchar(255)	○		送信元のマッチ条件（正 規表現）
subject_pattern	varchar(255)			件名のマッチ条件（販促 メールの除外に使う）
event_type	varchar(20)	○		生成する決済イベント種 別（3.6 参照）
payment_method_id	bigint		FK → payment_methods.id (SET NULL)	決済手段が固定の場合
body_format	varchar(10)	○	default text	text / html
extraction_rules	jsonb	○		各項目の抽出ルール（下 記）
is_multi_event	boolean	○	default false	1 通から複数イベントを生 成するか
priority	integer	○	default 100	評価順（小さいほど優先）
is_active	boolean	○	default true	有効フラグ
created_by	varchar(20)	○	default user	system / user / llm （LLM 生成をレビュー済 みか追跡）
created_at	timestampz	○		作成日時
updated_at	timestampz	○		更新日時

extraction_rules の構造例

```
{
  "occurred_at": { "regex": "ご利用日時: (.+?)\\n", "format": "2006/01/02
15:04" },
  "amount":      { "regex": "ご利用金額: ([0-9,]+)円" },
  "merchant":    { "regex": "ご利用先: (.+?)\\n" },
  "card_last4":  { "regex": "カード番号: \\*+([0-9]{4})" }
}
```

制約

- `UNIQUE(user_id, name)`

インデックス

インデックス名	対象カラム	用途
idx_parser_templates_match	(is_active, priority)	判定時の評価順取得
idx_parser_templates_from	from_pattern	送信元からのテンプレート検索

設計判断: 抽出ルールを列に展開せず `jsonb` にした理由は、送信元ごとに抽出したい項目が異なり（カード下 4 桁がある / ない、通貨がある / ない）、列で持つと `NULL` だらけになるうえ、項目追加のたびにマイグレーションが必要になるため。一方、判定条件（`from_pattern / subject_pattern`）は検索対象なので列で持つ。

3.6 payment_events

メール 1 通から抽出した決済イベント。**1 通のメールから複数行が生成されうる**（銀行の入出金まとめ通知、カード会社の月次確定通知など）。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
email_message_id	bigint	○	FK → email_messages.id (CASCADE)	抽出元メール
parser_template_id	bigint		FK → parser_templates.id (SET NULL)	使用したテンプレート
transaction_id	bigint		FK → transactions.id (SET NULL)	名寄せ結果。未処理時は NULL
link_type	varchar(20)			<code>auto / manual</code> （名寄せの経路）
link_score	numeric(5,2)			自動名寄せ時のスコア
linked_at	timestamptz			名寄せ日時
event_type	varchar(20)	○		イベント種別（下表）
occurred_at	timestamptz	○		決済日時
amount_minor	bigint	○		金額（最小単位）
currency	char(3)	○	default <code>JPY</code>	通貨コード
amount_jpy_minor	bigint	○		円換算額（最小単位）

カラム名	型	必須	制約	説明
raw_merchant_text	varchar(255)			生の店舗名文字列
merchant_id	bigint		FK → merchants.id (SET NULL)	正規化後の店舗
payment_method_id	bigint		FK → payment_methods.id (SET NULL)	決済手段
card_last4	varchar(4)			カード下 4 桁
normalized_at	timestampz			正規化完了日時
notified_at	timestampz			即時通知の送信済み日時。通知の冪等性を担保
created_at	timestampz	○		作成日時
updated_at	timestampz	○		更新日時

event_type の値

値	例	金額の確からしさ	即時通知
order_confirm	EC の注文確認	低	しない（まだ入金していない）
usage_notice	カード利用速報	中	する
settlement	カード確定明細	高	しない（速報時に通知済み）
receipt	サブスク領収書	高	する
bank_debit	銀行引落通知	高	する

インデックス

インデックス名	対象カラム	用途
idx_payment_events_user_occurred	(user_id, occurred_at DESC)	一覧・取引詳細での参照
idx_payment_events_reconcile	(user_id, amount_jpy_minor, occurred_at)	名寄せ（P5）の候補生成。金額一致 + 日時近接で絞る専用インデックス
idx_payment_events_transaction	transaction_id	取引に紐づくイベントの取得
idx_payment_events_unnotified	(user_id, notified_at)	未通知イベントの抽出（部分インデックス: WHERE notified_at IS NULL ）
idx_payment_events_email	email_message_id	メールからのイベント取得

3.7 transactions

名寄せ後の実取引。**1回の支出 = 1行**。

payment_events を参照するのではなく **自前のフィールドを持つ**。名寄せロジックを変更して全件再計算しても、ユーザの手修正が失われないようにするため。

現金など通知手段のない決済は **payment_events** を経由せず直接作成されるため、**紐づく payment_events が 0 件の行も正常**である。

このテーブルのみ論理削除 (deleted_at) とする。理由は下記の削除方針を参照。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	取引 ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
occurred_at	timestamptz	○		決済日時
amount_minor	bigint	○		金額（最小単位）
currency	char(3)	○	default JPY	通貨コード
amount_jpy_minor	bigint	○		円換算額（集計はこの列を使う）
merchant_id	bigint		FK → merchants.id (SET NULL)	店舗
category_id	bigint		FK → categories.id (SET NULL)	カテゴリ
payment_method_id	bigint		FK → payment_methods.id (SET NULL)	決済手段
category_source	varchar(20)	○	default default	rule / place_type / user / default （ユーザ修正をルールで上書きしないため）
source	varchar(20)	○		email / manual
status	varchar(20)	○	default confirmed	pending （速報のみ） / confirmed （確定明細あり）
is_user_edited	boolean	○	default false	true の行は再計算の対象外にする
is_possible_duplicate	boolean	○	default false	名寄せスコアが中程度で、ユーザ確認待ち

カラム名	型	必須	制約	説明
merged_into_id	bigint		FK → transactions.id (SET NULL)	マージで統合された先の取引ID（自己参照）。NULL 以外はマージ済みを意味する
note	text			メモ
deleted_at	timestampz			論理削除の実施日時。NULL の行のみ有効（下記の削除方針を参照）
created_at	timestampz	○		作成日時
updated_at	timestampz	○		更新日時

インデックス

インデックス名	対象カラム	用途
idx_transactions_user_occurred	(user_id, occurred_at DESC)	取引一覧の主軸（期間絞り込み・新着順）。部分インデックス: WHERE deleted_at IS NULL
idx_transactions_user_category_occurred	(user_id, category_id, occurred_at)	月次サマリのカテゴリ別集計。部分インデックス: WHERE deleted_at IS NULL
idx_transactions_merchant	merchant_id	店舗別集計・サブスク検出
idx_transactions_review	(user_id, is_possible_duplicate)	要確認キューの取得（部分インデックス: WHERE is_possible_duplicate AND deleted_at IS NULL ）
idx_transactions_merged	merged_into_id	マージ解除時に統合元を引く（部分インデックス: WHERE merged_into_id IS NOT NULL ）

参照系のクエリは常に **deleted_at IS NULL** で絞るため、主要インデックスは この条件を付けた **部分インデックス** にしている。削除済みの行が索引に載らず、 通常の一覧・集計が削除件数に影響されない。

削除方針

transactions は**論理削除**（**deleted_at** に日時を入れる）とする。 **論理削除はこのテーブルのみ**で、他のテーブルは物理削除のままとする。

操作	実行内容
ユーザによる削除	<code>deleted_at = now()</code>
マージ（統合元）	<code>deleted_at = now()</code> かつ <code>merged_into_id = <統合先の取引 ID></code>
マージ解除	<code>deleted_at</code> と <code>merged_into_id</code> を NULL に戻し、決済イベントを統合元へ付け替える

参照時は常に `deleted_at IS NULL` で絞る。

理由 1: 物理削除だと削除した取引が復活する

取引を物理削除すると `payment_events.transaction_id` が SET NULL となり、イベントが「未名寄せ」の状態に戻る。次の名寄せバッチ（P5）がそれを拾い、**同じ取引が再生成されてしまう**。論理削除なら行が残って決済イベントを保持し続けるため、バッチは「処理済み」と判断でき、復活しない。

理由 2: マージ解除が成立する

統合元を物理削除すると、`unmerge` で元に戻せない。イベントを NULL に戻せても、削除した取引行とそこに入っていたユーザの手修正（メモ・カテゴリ）は復元できないためである。

`merged_into_id` に統合先を記録して論理削除しておけば、両方の列を NULL に戻すだけで復元できる。

理由 3: 誤削除からの復旧

家計簿は取り返しのつかない削除を避けたいデータであり、「元に戻す」を提供できる余地を残す。

代償: 参照クエリすべてに `deleted_at IS NULL` が必要になり、付け忘れると削除済みの取引が画面に出る。GORM の `gorm.DeletedAt` 型を使うと自動で付与されるため、この漏れを防ぐ。

他テーブルに広げない理由: `categories` などのマスタを論理削除にすると、削除済みの行が `UNIQUE(user_id, name)` を占有し、同名で作直せなくなる。またマスタ系の FK はすでに SET NULL のため、物理削除しても不整合は起きない。

3.8 merchants

正規化済みの店舗マスタ。**1行 = 1支店**とし、`brand_name` でチェーンを束ねる。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	店舗 ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
name	varchar(255)	○		正規化済みの表示名（例: セブン-イレブン渋谷道玄坂店）

カラム名	型	必須	制約	説明
brand_name	varchar(100)			チェーン名（例: セブン-イレブン）
is_online	boolean	○	default false	EC などオンライン決済か
address	varchar(255)			住所
latitude	numeric(9,6)			緯度
longitude	numeric(9,6)			経度
place_id	varchar(255)			Places API の場所 ID
place_types	varchar(255)			店舗種別（カンマ区切り。カテゴリ推定に使う）
geocode_status	varchar(20)	○	default pending	pending / success / not_found / skipped
geocoded_at	timestamptz			ジオコーディング実施日時
created_at	timestamptz	○		作成日時
updated_at	timestamptz	○		更新日時

制約

- UNIQUE(user_id, name)

インデックス

インデックス名	対象カラム	用途
idx_merchants_user_brand	(user_id, brand_name)	ブランド単位の集計・カテゴリ規則の適用
idx_merchants_geocode_queue	geocode_status	未ジオコーディング店舗の抽出（部分インデックス: WHERE geocode_status = 'pending'）

3.9 merchant_aliases

店舗名の表記ゆれを **merchants** に対応づける。「アマゾン ジャパン」「AMAZON.CO.JP」「AMZN Mktp JP」が同じ店舗を指すことを表現する。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ

カラム名	型	必須	制約	説明
merchant_id	bigint	○	FK → merchants.id (CASCADE)	対応する店舗
alias	varchar(255)	○		正規化後の照合キー
raw_sample	varchar(255)			元の生文字列の例（デバッグ用）
source	varchar(20)	○	default auto	auto / user （ユーザが手動で紐づけたか）
created_at	timestampz	○		作成日時

制約

- **UNIQUE(user_id, alias)**: 同じ表記が複数店舗を指さないようにする。この制約の自動生成インデックスが、正規化（P4）の主要ルックアップも兼ねる。メール 1 通ごとに必ず引くため最も呼ばれる経路だが、専用インデックスは不要。

インデックス

インデックス名	対象カラム	用途
idx_merchant_aliases_merchant	merchant_id	店舗に紐づく表記ゆれ一覧

3.10 payment_methods

決済手段のマスタ。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
name	varchar(100)	○		表示名（例: 楽天カード、PayPay、現金）
kind	varchar(20)	○		credit_card / qr / bank / cash / other
issuer	varchar(100)			発行会社
card_last4	varchar(4)			カード下 4 桁（メールからの決済手段特定に使う）
is_active	boolean	○	default true	利用中フラグ
created_at	timestampz	○		作成日時
updated_at	timestampz	○		更新日時

制約

- `UNIQUE(user_id, name)`

インデックス

インデックス名	対象カラム	用途
idx_payment_methods_last4	(user_id, card_last4)	メール中の下 4 桁から決済手段を特定

3.11 categories

支出カテゴリのマスタ。親子関係を持てる。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	カテゴリ ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
name	varchar(50)	○		カテゴリ名（例: 食費、交通費）
parent_id	bigint		FK → categories.id (SET NULL)	親カテゴリ
sort_order	integer	○	default 0	表示順
is_system	boolean	○	default false	初期作成カテゴリか（ 未分類 を含む）
created_at	timestampztz	○		作成日時
updated_at	timestampztz	○		更新日時

制約

- `UNIQUE(user_id, name)`

インデックス

インデックス名	対象カラム	用途
idx_categories_user	(user_id, sort_order)	カテゴリ一覧の表示

3.12 category_rules

カテゴリ自動分類のルール。**ユーザが取引のカテゴリを修正すると、ここに upsert される**（学習）。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID

カラム名	型	必須	制約	説明
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
category_id	bigint	○	FK → categories.id (CASCADE)	適用するカテゴリ
match_type	varchar(20)	○		merchant / brand / place_type / keyword
match_value	varchar(255)	○		照合値（店舗 ID、ブランド名、店舗種別、キーワード）
priority	integer	○	default 100	評価順（小さいほど優先）
source	varchar(20)	○	default user	user / default
created_at	timestamptz	○		作成日時
updated_at	timestamptz	○		更新日時

制約

- **UNIQUE(user_id, match_type, match_value)**: 同じ条件で複数のカテゴリを指さないようにする。この制約の自動生成インデックスが、分類（P6）時の照合も兼ねる。

インデックス

インデックス名	対象カラム	用途
idx_category_rules_priority	(user_id, priority)	評価順の取得

設計判断: match_type = brand を用意したことで、「セブン-イレブン = 食費」を 1 回教えれば全支店に適用される。place_type は Places API から得た店舗種別で、一度も来店したことのない店舗でも初回から分類できる。

3.13 notification_settings

通知設定。ユーザ 1 人につき 1 行。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE), UNIQUE	所有ユーザ
channel	varchar(20)	○	default email	email / slack
email_to	varchar(255)			配信先メールアドレス

カラム名	型	必須	制約	説明
slack_webhook_encrypted	text			暗号化した Slack Webhook URL
instant_enabled	boolean	○	default true	即時通知の ON / OFF
instant_min_amount_minor	bigint	○	default 0	この金額未満は即時通知しない
quiet_hours_start	time			静音時間帯の開始
quiet_hours_end	time			静音時間帯の終了
budget_alert_enabled	boolean	○	default true	予算超過通知の ON / OFF
monthly_summary_enabled	boolean	○	default true	月次サマリの ON / OFF
monthly_summary_send_at	time	○	default 08:00	月次サマリの配信時刻
created_at	timestampzt	○		作成日時
updated_at	timestampzt	○		更新日時

制約

- **UNIQUE(user_id)**

3.14 notifications

通知の送信履歴。即時通知・予算超過通知・月次サマリの **3 種類を 1 テーブル** で扱う。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
kind	varchar(20)	○		instant / budget_alert / monthly_summary
channel	varchar(20)	○		email / slack
payment_event_id	bigint		FK → payment_events.id (SET NULL)	即時通知の対象イベント
dedupe_key	varchar(255)	○		二重送信を防ぐキー （下記）
subject	varchar(255)			件名

カラム名	型	必須	制約	説明
body	text			本文
status	varchar(20)	○	default pending	pending / sent / failed
sent_at	timestampz			送信日時
error_message	text			送信失敗の内容
created_at	timestampz	○		作成日時

dedupe_key の設計

kind	dedupe_key の例	意味
instant	instant:event:12345	同じ決済イベントで 2 回通知しない
budget_alert	budget:2026-09:cat:3:80	同じ月・同じカテゴリ・同じ閾値で 2 回通知しない
monthly_summary	monthly:2026-09	同じ月のサマリを 2 回送らない

制約

- **UNIQUE(user_id, dedupe_key)** ← 3 種類すべての二重送信をこの 1 制約で防ぐ。バッチが 1 分ごとに再実行されても安全。

インデックス

インデックス名	対象カラム	用途
idx_notifications_user_created	(user_id, created_at DESC)	通知履歴画面の表示
idx_notifications_pending	status	送信リトライ対象の抽出（部分インデックス: WHERE status <> 'sent' ）

3.15 subscriptions（拡張）

検出したサブスクリプション。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
merchant_id	bigint	○	FK → merchants.id (CASCADE)	店舗

カラム名	型	必須	制約	説明
amount_minor	bigint	○		課金額
currency	char(3)	○	default JPY	通貨コード
cycle	varchar(20)	○		monthly / yearly / weekly
occurrence_count	integer	○		検出に使った課金回数
first_charged_at	timestampz	○		初回課金日
last_charged_at	timestampz	○		最終課金日
next_expected_at	timestampz			次回課金の予測日
status	varchar(20)	○	default active	active / suspected_stopped / cancelled
detected_at	timestampz	○		検出日時
created_at	timestampz	○		作成日時
updated_at	timestampz	○		更新日時

制約

- UNIQUE(user_id, merchant_id, amount_minor, cycle)

インデックス

インデックス名	対象カラム	用途
idx_subscriptions_user_status	(user_id, status)	継続中サブスク一覧の取得
idx_subscriptions_next_expected	next_expected_at	停止疑いの判定

3.16 budgets（拡張）

予算設定。category_id が NULL の行は全体予算を表す。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
category_id	bigint		FK → categories.id (CASCADE)	NULL は全体予算
period	varchar(10)	○	default monthly	予算の期間
amount_minor	bigint	○		予算額

カラム名	型	必須	制約	説明
alert_thresholds	varchar(50)	○	default 80,100	通知する到達率（%、カンマ区切り）
is_active	boolean	○	default true	有効フラグ
created_at	timestampz	○		作成日時
updated_at	timestampz	○		更新日時

制約

- UNIQUE(user_id, category_id, period)

3.17 monthly_summaries（拡張）

月次サマリの生成・配信記録。

カラム名	型	必須	制約	説明
id	bigint	○	PK, auto increment	ID
user_id	bigint	○	FK → users.id (CASCADE)	所有ユーザ
year_month	char(7)	○		対象月（例: 2026-09）
total_amount_minor	bigint	○		当月合計
prev_total_amount_minor	bigint			前月合計（前月比の算出用）
breakdown	jsonb	○		カテゴリ別内訳
active_subscription_count	integer	○	default 0	継続中サブスク数
notification_id	bigint		FK → notifications.id (SET NULL)	配信した通知
generated_at	timestampz	○		生成日時
created_at	timestampz	○		作成日時

制約

- UNIQUE(user_id, year_month): 同じ月のサマ리를二重生成しない

4. インデックス設計の考え方

観点	該当インデックス	理由
画面の 主要導 線	<code>idx_transactions_user_occurred</code>	取引一覧は「期間で絞って新着順」が基本操作
集計ク エリ	<code>idx_transactions_user_category_occurred</code>	月次サマリのカテゴリ別内訳
アルゴ リズム 専用	<code>idx_payment_events_reconcile</code>	名寄せの候補生成（金額一致 + 日時近接）を支えるための複合インデックス
バッチ のキュー 取得	<code>idx_email_messages_queue</code> , <code>idx_merchants_geocode_queue</code> , <code>idx_payment_events_unnotified</code>	各工程が「未処理のものだけ」を高速に拾うため。該当行が少ないので部分インデックスにする
正規化 のルック アップ	<code>merchant_aliases</code> の <code>UNIQUE(user_id, alias)</code>	メール 1 通ごとに必ず引くため最も呼ばれるが、制約の自動生成インデックスで賄える
可観測 性	<code>idx_email_messages_from</code>	送信元ごとの受信件数を集計し、途絶を検知する

制約が自動生成するインデックスと重複させない

PostgreSQL は `PRIMARY KEY` と `UNIQUE` に対して一意インデックスを自動生成する。複合 `UNIQUE(A, B)` は `WHERE A = ?` の検索にも使える（左端プレフィックス）ため、**これらと重なる専用インデックスは定義しない**。重複して定義すると、容量を無駄に使ううえ、書き込みのたびに同じ内容のインデックスを 2 つ更新することになる。

一方、`FOREIGN KEY` にはインデックスが自動生成されない点には注意する。参照先（マスタ）の行を削除する際に参照元の全表走査が発生するが、本サービスではマスタの削除頻度が低いいため、専用インデックスは設けていない。

5. 主な設計判断

- 3 層分割（`email_messages` → `payment_events` → `transactions`）** 生データを保持することで、抽出・名寄せロジックの変更時に全件再計算できる。
- `email_messages` 1 - N `payment_events`** 銀行のまとめ通知など、1 通のメールから複数の決済が読み取れるケースがあるため。
- 名寄せに中間テーブルを使わなかった** 1 つの決済イベントが属する取引は 1 つだけなので、関係は N - 1 である。中間テーブルは不要と判断し、`payment_events.transaction_id` の外部キーで表現した。マージ解除は、この列と統合元の `deleted_at / merged_into_id` を戻すことで実現する（3.7 参照）。

4. **transactions** が自前のフィールドを持つ 代表イベントを参照する設計にすると、再計算時にユーザーの手修正が失われる。 **is_user_edited** で再計算対象から除外する。
5. **payment_events.event_type** を中心列にした 金額の確からしさ判定（名寄せ後にどの金額を採用するか）と、 即時通知の対象を絞る判定の両方で使う。
6. **merchants** を支店単位にし、 **brand_name** を列で持った 地図表示は支店単位、集計とカテゴリ規則はブランド単位で行いたいため。 **brands** テーブルを分けなかったのは、テーブル数と実装期間を考慮したスコープ判断。
7. **notifications.dedupe_key** で 3 種類の通知の冪等性をまとめた バッチが 1 分間隔で再実行されるため、二重送信の防止が必須。 **UNIQUE(user_id, dedupe_key)** の 1 制約で 3 種類すべてをカバーする。
8. **parser_templates.extraction_rules** を **jsonb** にした 送信元ごとに抽出項目が異なるため。判定条件は検索対象なので列で持つ。
9. 金額を **bigint** の最小単位で保持 浮動小数点の誤差を避ける。外貨は「原通貨 + 原金額 + 円換算額」で保持する。
10. マルチテナント設計 運用はシングルユーザだが、全業務テーブルに **user_id** を持たせた。後から追加するコストが高いため。
11. メール取得をポーリングにし、 **Push 通知への移行余地を残した** 初期実装は 1 分間隔のポーリング。 **Push (users.watch + Cloud Pub/Sub)** は **watch** の更新バッチなど主題外の運用コストが大きいため見送った。ただし **Push** が通知するのも **historyId** で **sync_cursor** の意味は変わらないため、 **sync_jobs.trigger** に **push** を追加するだけで移行できる設計にしてある（3.3 参照）。
12. **sync_jobs** を間引きつつ、死活監視は **last_synced_at** で担保した 新着なしの実行を記録しないと「新着なし」と「バッチ停止」が区別できなくなる。成功時に 0 件でも **mail_accounts.last_synced_at** を更新することで、行数を増やさずに停止検知を成立させた。
13. 連携解除を物理削除にしなかった **email_messages** は **CASCADE** で **mail_accounts** にぶら下がるため、連携解除で行を削除するとメールの生データまで失われ、再計算ができなくなる。連携解除は **status = 'disabled' + トークン破棄** で表現し、全データの削除は「退会（**users** の削除）」という別操作に分けた（3.2 参照）。
14. **transactions** だけを論理削除にした 物理削除すると **payment_events.transaction_id** が **SET NULL** となり、次の名寄せバッチが同じ取引を再生成する（削除した取引が復活する）。 **deleted_at** で行を残して決済イベントを保持させることでこれを防いだ。あわせて **merged_into_id** を持たせ、統合元を残すことでマージ解除も成立させている。マスタ系は **UNIQUE** 制約を占有してしまうため論理削除にせず、例外はこのテーブルのみとした（3.7 参照）。

6. 判断に迷った点（メンタリングで相談したい）

1. **sync_jobs** の記録粒度 — 「新着なしの実行は記録せず、死活監視は **last_synced_at** の更新で行う」方針とした（3.3 参照）。行数は抑えられるが実行履歴が歯抜けになるため、障害調査の際に不都合が出ないか
2. **transactions.status** (**pending / confirmed**) を持つべきか — 利用速報と確定明細で金額が変わる問題への対処として適切か

3. **merchants** を支店単位にした判断 — **brands** テーブルを分けるべきか
4. **email_messages.body_text / body_html** の保持 — 再解析のために必要だが、プライバシーと容量の懸念がある。3層構造により**このテーブルを削除しても家計簿データは残る**（失うのは再計算とトレーサビリティのみ）ため、「本文はNか月経過後に削除する」といった保持期間を設ける余地はある。どの程度が妥当か
5. 部分インデックスの多用 — PostgreSQL 前提の設計になっているが問題ないか